

Week 6 — Instruction Sets and Machine Code

Target time: approximately 3 hours

Suggested rhythm: 75 minutes building the model · 35 minutes comparing real instruction sequences · 45 minutes in the C/disassembly lab · 35 minutes designing and testing Cedar-16 · 10 minutes explaining without notes

A CPU contains gates that can add, compare, move bits, and choose a next state. How does a compiler ask that machinery to perform one particular operation—and how can a different CPU understand the same request?

Why this week exists

The previous path assembled the ingredients of a minimal processor:

gates → arithmetic and selection → registers → datapath + control
→ fetch → decode → execute

There is still a missing agreement.

Imagine buying a new processor whose designer tells you:

“It has registers, an ALU, and a very fast decoder.”

That does not tell a compiler how to generate a program for it. Which bit pattern requests addition? How many registers can software name? Can an arithmetic instruction read memory directly? How is a branch target encoded? What happens when arithmetic overflows? What does an invalid bit pattern do?

Hardware and software need a durable contract. That contract is the **instruction set architecture**, or **ISA**.

This week we will make that contract concrete at three levels:

ISA specification	written rules and programmer-visible behavior
machine-code bytes	encoded instructions in an object file or memory
processor implementation	decoder and execution machinery that obey the rules

Those are related, but they are not the same object and they are not stored in one place called “the ISA.”

By the end, the central mastery question should feel inevitable:

Why can two CPUs implement the same ISA differently?

Part I — The historical problem: programs needed a stable language

Early machines often exposed the machine too directly

Before stored-program computers became normal, “programming” could mean rewiring a machine, setting switches, or arranging external media. The operation and the physical setup were tightly coupled. Changing the problem could mean changing the machine's configuration.

The stored-program idea made a powerful move: represent instructions as patterns in memory, alongside data. The control unit could fetch one pattern, interpret it, perform an operation, and fetch another.

That solved one problem and created the next:

Which patterns mean which operations?

A bit pattern has no intrinsic command hidden inside it. A designer must assign an interpretation, document the assignment, and build a processor that behaves accordingly.

Suppose an early design chooses:

```
0001 ddd aaa bbb 000
```

and declares it to mean:

```
ADD register[a] and register[b], place the result in register[d]
```

The declaration alone does not execute anything. The physical decoder must recognize `0001`, select register ports, command the ALU to add, enable a destination write, and advance the program counter. Conversely, the circuitry alone is not a usable software interface until its behavior is specified.

This separation became increasingly valuable as computer families evolved. Software investments were expensive. Customers wanted newer, faster machines without rewriting every program. A compatible architecture could remain stable while vacuum tubes gave way to transistors, narrow datapaths widened, caches appeared, pipelines deepened, and execution became speculative and out of order.

Compatibility is therefore not historical clutter by definition. It is an engineering asset with costs and benefits.

Architecture became a boundary

The ISA boundary says, roughly:

```
software may rely on these visible operations and results
hardware may choose how to produce those results
```

This boundary lets:

- a compiler target an architecture rather than one transistor layout;

- an operating system manage defined registers, privilege states, and exceptions;
- multiple processor vendors or generations run compatible binaries;
- hardware designers improve performance without changing every program;
- simulators and virtual machines implement the same visible behavior in software.

The boundary is not perfectly simple. Real ISAs contain optional extensions, implementation-defined behavior, privileged state, memory-ordering rules, and versioned features. Software may query which features exist. But the central idea remains: the architecture specifies the machine that software is allowed to see.

Part II — ISA versus microarchitecture

What belongs to the ISA?

An ISA normally specifies programmer-visible matters such as:

- available instruction encodings and their semantics;
- architectural registers and their widths;
- supported data types and arithmetic behavior;
- how instructions name register, memory, and immediate operands;
- address calculation and the architectural address space;
- program-counter behavior and control transfers;
- visible status flags or control/status registers;
- exceptions caused by invalid operations or access conditions;
- privilege levels and privileged instructions;
- atomic operations and memory-ordering rules;
- sometimes optional extensions that software can detect.

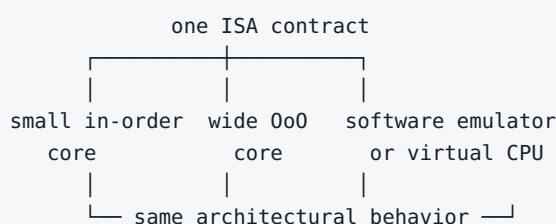
If two conforming processors execute the same valid architectural program from the same architectural starting state, software expects the specified architectural outcome.

What belongs to a microarchitecture?

A **microarchitecture** is one particular internal organization used to implement an ISA. It may choose:

- pipeline depth;
- number and kind of execution units;
- in-order or out-of-order scheduling;
- cache sizes, associativity, and replacement details;
- branch predictor design;
- instruction fetch and decode width;
- internal physical-register count and register-renaming scheme;
- whether an instruction uses hardwired control, several internal operations, or a microcode-assisted sequence;
- power-management and clocking strategies.

These choices affect speed, power, area, cost, and sometimes timing-visible side effects. They do not normally change the required architectural answer.



A compact low-power x86-64 core and a wide server x86-64 core can decode the same architectural `ADD` instruction while using very different pipelines, caches, predictors, and internal schedules. Likewise, many different RISC-V implementations can implement the same selected base ISA and extensions.

Architectural state versus hidden working state

Architectural state is the state the ISA says software can observe: for example, the program counter, general-purpose registers, and defined status state.

Microarchitectural state helps an implementation do its work:

```
branch-predictor entries
cache replacement metadata
reorder-buffer entries
physical-register mappings
decoded-instruction queues
```

Ordinary instructions do not name those structures directly. They may affect performance and, as security research has taught us, hidden state can sometimes leave observable timing traces. That does not turn predictor tables into architectural registers. It shows that “not architecturally visible” is not identical to “physically nonexistent” or “incapable of side effects.”

Pause and explain

For each item, classify it as ISA, microarchitecture, or possibly both at different levels:

1. `RAX` is a 64-bit programmer-visible register.
2. The L1 data cache is 48 KiB.
3. A particular opcode performs integer addition.
4. The processor can issue six internal operations in one cycle.
5. A misaligned load either succeeds or raises a specified exception.
6. A branch predictor guesses the next address.

The useful test is:

Must normal software know this to produce the architecturally correct result, or may the processor designer change it while preserving the contract?

Part III — What is a machine instruction?

A **machine instruction** is an ISA-defined operation represented by an instruction encoding. When fetched and successfully decoded, it requests a defined architectural state transition.

Consider an abstract instruction:

```
ADDI r3, r2, 12
```

Its fields have different jobs:

- **opcode**: selects the operation family, here add-immediate;
- **destination operand**: `r3`, where the architectural result goes;
- **source operand**: `r2`, one input value;
- **immediate**: `12`, a constant carried inside the instruction encoding.

A possible fixed-width encoding might be:

```
bits: 15          12 11          9 8          6 5          0
      +-----+-----+-----+-----+
      | opcode=0101| rd = 011 | rs1=010 | immediate=001100 |
      +-----+-----+-----+-----+
```

The immediate bits occupy space that could otherwise name more registers, more operations, or a larger constant. Encoding is resource allocation inside a finite number of bits.

Opcode does not always mean “one neat field”

Teaching diagrams often show one contiguous opcode field. That is useful, but real encodings can be more involved.

An instruction's operation may be selected by a combination of:

- a primary opcode;
- extension fields;
- prefix bytes;
- function fields;
- bits that also influence operand size or addressing.

RISC-V base instruction formats use a 7-bit major opcode plus fields such as `funct3` and `funct7` to distinguish operations. x86 encodings may use legacy, REX, VEX, or EVEX prefixes, one- or multi-byte opcode maps, ModR/M and SIB bytes, displacements, and immediates.

The conceptual word “opcode” still helps: it is the encoded information that selects the operation. Do not insist that every architecture puts all of that information in one rectangle.

Operands answer “operation on what?”

Common operand sources are:

1. **register operands** — names select values in architectural registers;
2. **memory operands** — an effective address identifies a memory location;
3. **immediate operands** — constant bits are embedded in the instruction;

4. **implicit operands** — the ISA definition names a value not written explicitly in assembly syntax.

Examples in x86-64 AT&T syntax:

```
addq %rcx, %rax      # register source, register destination
addq $7, %rax        # immediate source, register destination
addq (%rdi), %rax    # memory source, register destination
```

AT&T syntax writes source before destination and prefixes registers with `%` and immediates with `$`. Intel syntax commonly reverses operand order and uses different notation. Syntax is an assembler language convention. The CPU sees the encoded bytes, not `%`, `$`, commas, or source comments.

Immediate values are not tiny memory variables

In:

```
addq $7, %rax
```

the value `7` is encoded as part of the instruction. Fetching the instruction brings in those immediate bits. The processor does not first load `7` from a data object at address 7.

Immediate width forces tradeoffs. A small field makes common constants compact but cannot directly represent every value. Architectures provide strategies such as:

- sign-extend a small immediate;
- use several instructions to construct a large constant;
- load a constant from memory;
- provide special upper-immediate or literal-loading forms.

Sign extension is an architectural rule, not a guess made by the decoder.

Part IV — Encoding is a constrained design problem

Fixed-length instructions

A fixed-width design makes instruction boundaries easy to find:

```
32-bit instruction | 32-bit instruction | 32-bit instruction
```

Benefits can include:

- regular fetch alignment;
- simpler parallel field extraction;
- predictable next sequential address;
- a relatively regular decoder.

Costs include:

- simple operations still occupy the full width;
- large register sets, large immediates, and many opcodes compete for fields;

- some operations need multiple instructions;
- code density may suffer.

The base RISC-V integer ISA uses fixed 32-bit instructions. The optional compressed extension adds 16-bit encodings for common operations, so a real RISC-V instruction stream need not be purely 32-bit. “RISC-V is fixed-width” is therefore incomplete unless the chosen ISA profile excludes compressed instructions.

Variable-length instructions

x86 instructions are variable length, architecturally from 1 through 15 bytes. A conceptual layout is:

```
[prefixes] [opcode] [ModR/M] [SIB] [displacement] [immediate]
```

Not every instruction has every part.

Benefits include:

- frequent simple operations can be compact;
- old encodings can coexist with newer extension mechanisms;
- operands and addressing forms can be described flexibly;
- code density can be good for many workloads.

Costs include:

- finding several instruction boundaries in parallel is harder;
- decode hardware is more complex;
- instruction length is not known from one universal field;
- legacy constraints consume encoding space and design effort.

Neither scheme wins every metric. A regular encoding can simplify parts of a front end but require more instructions or bytes for a task. A rich variable-length encoding can express work compactly but demand sophisticated fetch and decode machinery.

There is no free encoding field

Suppose Cedar-16 has only 16 bits per instruction:

```
4-bit opcode + 3-bit rd + 3-bit rs1 + 3-bit rs2 + 3 spare bits
```

The 3-bit register fields name only eight registers. To name 32 registers each field would need 5 bits, adding six bits across three operands. Something else must shrink, move to another format, or require a wider instruction.

Ask these questions of any proposed format:

- How many operations can it distinguish?
- How many registers can it name?
- How large is its immediate or branch range?
- Can every bit pattern be decoded unambiguously?
- Are some patterns reserved for future expansion?
- How does a decoder find the next instruction?

- What common operations require more than one instruction?

An instruction encoding is not just a binary translation exercise. It is a budget.

Part V — Register operands, memory operands, and load/store design

Why registers matter

Registers provide a small, directly named working set near execution units. An instruction can encode a register number in a few bits, and the microarchitecture can build fast register-file access around those names.

Memory is much larger and addressed differently. A memory operand normally requires an **effective address** calculation, then access through the memory system. At the ISA level we describe the address and result. Later memory hierarchy lessons will explain why latency can vary dramatically.

Load/store architectures

In a load/store design, ordinary arithmetic operates on registers. Memory is accessed with explicit load and store instructions.

A conceptual RISC-V sequence for `*p += 7`, with `p` in `a0`, is:

```
ld    t0, 0(a0)      # memory → register
addi  t0, t0, 7       # register arithmetic
sd    t0, 0(a0)      # register → memory
```

RISC-V integer arithmetic instructions do not take a general memory operand. This regularity makes data movement explicit.

x86 permits memory operands in many operations

An equivalent x86-64 sequence, with `p` in `rdi`, can be:

```
addq $7, (%rdi)      # read-modify-write the addressed 64-bit memory object
```

Architecturally this is one instruction with a memory destination and an immediate source. Do not infer that it takes one cycle, one internal action, or one memory transaction in every situation.

Also do not infer that x86 allows arbitrary combinations. Most general-purpose x86 instructions allow at most one explicit memory operand. An instruction such as a general `mov` cannot normally copy from one arbitrary memory location directly to another arbitrary memory location; a register is usually needed between them. Special string instructions have defined implicit memory operands, which is another reminder that real ISAs resist one-sentence summaries.

Instruction count is not performance

The example has one x86 instruction and three RISC-V instructions. That alone does not prove the x86 sequence is three times faster.

Performance depends on matters such as:

- the actual implementations;
- instruction bytes and front-end bandwidth;
- decoded internal operations;
- dependency chains;
- cache hits or misses;
- available execution ports;
- surrounding code and scheduling;
- branch behavior and speculation.

Instruction count is useful evidence. It is not a complete performance model.

Modern x86 and micro-operations: the careful version

Many modern high-performance x86 implementations decode architectural x86 instructions into internal **micro-operations**, often shortened to μ ops. Those μ ops can be renamed, scheduled, and executed by an out-of-order back end. Some complex instructions need several μ ops or microcode-assisted flows; some simple instructions map economically; implementations can also fuse certain operations in specific circumstances.

The caveats matter:

1. μ ops are generally microarchitectural, not the x86 ISA exposed to normal application code.
2. Intel and AMD need not use identical μ ops or decoding machinery.
3. “x86 becomes RISC internally” is too vague to be a sound model. Internal operations are implementation-specific and are not simply RISC-V instructions.
4. A single architectural instruction is not guaranteed to be one μ op, and one source-level operation is not guaranteed to be one instruction.

The stable claim is:

The front end accepts architectural x86 instruction bytes; a particular microarchitecture may translate them into its own internal work while preserving architectural behavior.

Part VI — Addressing modes: how an instruction names memory

An **addressing mode** defines how an instruction obtains an operand or computes its effective address.

Register direct

```
addq %rcx, %rax
```

The encoded register fields name both operand locations.

Immediate

```
addq $12, %rax
```

The source value is carried in the instruction.

Base plus displacement

```
movl 12(%rdi), %eax
```

Conceptually:

```
effective address = RDI + 12  
load 4 bytes from that address
```

This is useful for a structure member at a known offset or a local object relative to a frame/base register.

Base plus scaled index plus displacement

x86-64 can express:

```
movl 8(%rdi,%rsi,4), %eax
```

Conceptually:

```
effective address = RDI + RSI × 4 + 8
```

Scale factors supported by this addressing form are 1, 2, 4, or 8. This maps naturally to indexed arrays and fields, though the compiler is free to choose another sequence.

PC-relative addressing

The effective address can be based on the current instruction location. This is useful for branches and position-independent references:

```
target = architecturally defined PC base + signed displacement
```

The exact PC base and scaling rule are ISA-specific. Do not assume every architecture uses the address of the branch instruction itself or measures the displacement in bytes.

RISC-V's regular memory address form

Base RISC-V integer loads and stores use a register base plus a signed 12-bit immediate offset. A more elaborate address is normally calculated in registers first.

Conceptually:

```
slli t0, a1, 3      # t0 = index × 8  
add  t0, a0, t0      # t0 = base + scaled index  
ld   t1, 16(t0)     # load at base + index×8 + 16
```

This is not a defect disguised as simplicity, nor is x86's richer mode a free win. They allocate complexity and encoding space differently.

Address calculation is not memory access

An effective address is an architectural value used to identify a memory location. Calculating it does not guarantee the access succeeds. Translation, permissions, alignment rules, and faults belong to the complete execution path.

This distinction will matter repeatedly:

```
instruction bytes specify addressing rule
      ↓
register values + encoded displacement produce effective address
      ↓
memory system attempts the access
      ↓
architectural result or defined exception
```

Part VII — RISC and CISC without caricatures

The labels **RISC** and **CISC** arose from real design debates, but slogans often erase the useful parts.

Tendencies associated with RISC designs

Historically common goals include:

- relatively regular instruction formats;
- load/store organization;
- many general-purpose registers;
- operations that are convenient to pipeline;
- letting compilers combine simpler instructions;
- reserving complex policy from the common hardware path.

RISC-V makes many of these choices visible: a small base ISA, extension structure, load/store integer operations, and regular base formats.

But “RISC means every instruction is simple and one cycle” is false. Modern RISC-family designs include floating-point, vector, atomic, cryptographic, and privileged operations. Loads can miss caches. Division can take many cycles. Implementations can be wide, speculative, and highly complex.

Tendencies associated with CISC designs

Historically common characteristics include:

- many instruction forms and addressing options;
- variable-length encodings;
- operations that can combine memory access with computation;
- strong compatibility across long-lived product families;
- instructions with substantial architectural behavior.

x86-64 exhibits these characteristics.

But “CISC means one giant instruction does the whole program” is false. Compilers usually select efficient subsets and sequences. Modern x86 cores use advanced pipelines and internal operations. Many common x86 instructions are ordinary moves, arithmetic operations, comparisons, and branches.

Better questions than “which philosophy won?”

Ask instead:

- How regular is decode?
- How dense is common code?
- How many architectural instructions are needed for this operation?
- What work does each instruction specify?
- How much compatibility must be preserved?
- How easily can implementations scale from tiny to aggressive?
- What does the compiler need to synthesize?
- What are the power, area, verification, and toolchain costs?

The boundary has also blurred. RISC architectures add extensions and compact encodings; x86 implementations translate and schedule internal operations. The architectural contracts remain meaningfully different even though implementation techniques cross historical categories.

Part VIII — Equivalent x86-64 and RISC-V sequences

These examples are conceptual but use real instruction forms. Assume the usual Unix-like calling conventions only to choose argument registers: the first pointer argument arrives in `rdi` on System V AMD64 and in `a0` on the RISC-V ELF psABI.

That sentence introduces an important distinction:

- the **ISA** defines instructions and architectural registers;
- the **ABI** defines software conventions such as argument registers, caller/callee-saved registers, stack alignment, object-file details, and system interfaces.

Another ABI could use the same ISA differently. We will study ABI mechanics in depth later.

Example A: load a 64-bit value and add seven

C intent:

```
uint64_t add_seven(const uint64_t *p)
{
    return *p + 7;
}
```

x86-64, AT&T syntax:

```
movq (%rdi), %rax
addq $7, %rax
ret
```

RV64I:

```
ld    a0, 0(a0)
addi  a0, a0, 7
ret                                # assembler pseudoinstruction for jalr x0, 0(ra)
```

The sequences are similar because both perform a load, addition, and return. The exact encodings are not similar, and `ret` is not necessarily a distinct hardware opcode in both assembler languages.

Example B: increment a value in memory

C intent:

```
void increment(uint64_t *p)
{
    *p += 1;
}
```

Plausible x86-64:

```
addq $1, (%rdi)
ret
```

Plausible RV64I:

```
ld    t0, 0(a0)
addi  t0, t0, 1
sd    t0, 0(a0)
ret
```

Important precision: neither sequence makes a concurrent non-atomic C update safe. “One x86 instruction” does not automatically mean an atomic read-modify-write visible to other cores; x86 uses specific locking semantics for that purpose. We are comparing operand models, not teaching concurrency yet.

Example C: indexed load

C intent:

```
uint64_t element(const uint64_t *base, size_t index)
{
    return base[index];
}
```

Plausible x86-64:

```
movq (%rdi,%rsi,8), %rax
ret
```

Plausible RV64I:

```
slli a1, a1, 3
add a0, a0, a1
ld a0, 0(a0)
ret
```

The x86 addressing mode carries scale information in the load. RISC-V constructs the address through register operations. This makes the design tradeoff visible, but still does not predict total runtime by counting lines.

Comparison checklist

For each pair, record:

1. architectural instruction count;
2. which instructions access memory;
3. where constants appear;
4. how the effective address is formed;
5. which register holds the return value by ABI convention;
6. which lines are assembler pseudoinstructions;
7. what cannot be concluded about cycles or μops .

Part IX — Where does the ISA actually exist?

This question deserves a precise layered answer.

1. The ISA exists as a specification

Intel, AMD, and RISC-V publish architecture manuals. These documents define encodings, behavior, exceptions, state, and extension rules. The specification is a contract expressed in documents and formal descriptions.

A PDF does not control the processor in real time. It communicates the rules to hardware designers, compiler writers, operating-system developers, verification engineers, and tool builders.

2. A program contains machine-code encodings

An ELF executable or object file contains byte sequences in sections such as `.text`. When loaded, executable mappings contain those bytes. A disassembler uses ISA decoding rules to render bytes as assembly text.

```
object/executable .text bytes
    ↓ loader maps bytes
process virtual memory contains instruction bytes
    ↓ fetch
processor receives bytes
```

Those bytes are **instances of encoded instructions**. The file does not contain the entire ISA specification.

3. A processor implements decode and behavior

The CPU front end fetches instruction bytes and identifies valid encodings. Control and execution structures cause the required architectural behavior. This implementation may include combinational logic, tables, caches of decoded work, sequencers, and microcode. The exact design is processor specific.

It is imprecise to say “the ISA is stored in the decoder.” The decoder implements part of the contract. Other parts involve execution units, architectural-state machinery, memory ordering, exception handling, and more.

4. Tools contain models of the encoding rules

Assemblers, compilers, disassemblers, emulators, debuggers, and binary translators also contain software representations of ISA rules:

```
assembler:    mnemonic + operands → bytes
disassembler: bytes → textual instruction
emulator:     bytes + state → modeled next state
compiler:     language operations → chosen instruction sequences
```

None of these is “the one place” where the ISA exists. They are artifacts or implementations aligned to the specification.

The sentence to keep

The ISA is a specification-level contract; machine code is its encoded program representation; decoders and execution machinery are physical implementations of its behavior.

Part X — Observe machine code in a real ELF file

Use:

`weekly/challenges/week-006-machine-code-lab.c`

The program deliberately contains:

- indexed loads from an integer array;
- widening arithmetic and multiplication;
- loops and conditional behavior;
- a `switch` with several result paths;
- separately visible functions for focused disassembly.

Step 1 — Predict before compiling

For `weighted_sum`, predict:

1. Which source operations require memory loads?
2. Which values could remain in registers through the loop?
3. Where must control return to repeat the loop?

4. What must happen when `index == count`?

For `count_above`, predict whether optimized code must contain a conditional branch for the source-level `if`. The correct prediction is not “yes because C has `if`.” A compiler may use any target sequence that preserves the language semantics.

Step 2 — Build with GCC and Clang

From the repository root:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -O2 -g \
    weekly/challenges/week-006-machine-code-lab.c -o /tmp/week6-gcc

clang -std=c17 -Wall -Wextra -Wpedantic -O2 -g \
    weekly/challenges/week-006-machine-code-lab.c -o /tmp/week6-clang

/tmp/week6-gcc
/tmp/week6-clang
```

Warnings are evidence. Do not suppress them merely to make the terminal quiet. Both compilers should build this source warning-clean.

Step 3 — Ask `readelf` what kind of object this is

```
readelf -h /tmp/week6-gcc
readelf -S /tmp/week6-gcc
readelf -sW /tmp/week6-gcc
```

Find:

- the `Class` and `Machine` fields in the ELF header;
- the `.text` section;
- symbols for `weighted_sum`, `count_above`, and `choose_operation`.

`readelf` is showing ELF structure and metadata. It is not primarily decoding the instruction stream.

Step 4 — Ask `objdump` to decode bytes

```
objdump -d -M intel --disassemble=weighted_sum /tmp/week6-gcc
objdump -d -M intel --disassemble=count_above /tmp/week6-gcc
objdump -d -M intel --disassemble=choose_operation /tmp/week6-gcc
objdump -s -j .text /tmp/week6-gcc
```

The `-d` output places instruction bytes beside disassembled text. Choose one instruction and mark:

```
virtual address | raw bytes | mnemonic | rendered operands
```

Then locate those bytes in the `.text` dump. The assembly rendering is not stored as those words in the executable. `objdump` reconstructs it by decoding bytes according to the selected architecture and syntax.

Step 5 — Change the compiler's choices

Build an unoptimized version:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g \
    weekly/challenges/week-006-machine-code-lab.c -o /tmp/week6-gcc-00

objdump -d -M intel --disassemble=count_above /tmp/week6-gcc-00
objdump -d -M intel --disassemble=count_above /tmp/week6-gcc
```

Compare:

- number of loads and stores;
- stack use;
- register reuse;
- loop shape;
- branch or branchless conditional handling;
- function size.

The C source did not prescribe one machine-code sequence. The compiler, optimization level, target ISA, ABI, and available extensions jointly constrain the generated program.

Step 6 — Compare GCC and Clang without declaring a winner

```
objdump -d -M intel --disassemble=weighted_sum /tmp/week6-gcc
objdump -d -M intel --disassemble=weighted_sum /tmp/week6-clang
```

Record differences. Different output is not evidence that one compiler broke the ISA. Run both programs and connect each sequence back to the required C result.

Optional: produce assembly without assembling

```
gcc -std=c17 -Wall -Wextra -Wpedantic -O2 -S \
    weekly/challenges/week-006-machine-code-lab.c -o /tmp/week6-gcc.s

clang -std=c17 -Wall -Wextra -Wpedantic -O2 -S \
    weekly/challenges/week-006-machine-code-lab.c -o /tmp/week6-clang.s
```

These `.s` files contain assembler text. They are not yet ELF machine-code executables. This distinction anticipates the later toolchain lesson:

```
C source → compiler-generated assembly → assembler → object bytes
           → linker → ELF executable → loader → memory → CPU fetch
```

Part XI — Build and challenge a fictional ISA

Use:

```
weekly/challenges/week-006-fictional-isa.py
```

It implements an encoder and decoder for **Cedar-16**, a deliberately tiny 16-bit teaching ISA. It does not emulate execution; its purpose is to make encoding choices inspectable.

The formats

```
R: [ opcode:4 | rd:3 | rs1:3 | rs2:3 | 000:3 ]  
I: [ opcode:4 | rd:3 | rs1:3 | signed immediate:6 ]  
B: [ opcode:4 | rs1:3 | rs2:3 | signed PC offset:6 ]  
J: [ opcode:4 | signed PC offset:12 ]
```

LOAD and **STORE** use I-shaped fields. In a store, the field occupying **rd** names a source register rather than a destination. Offsets are counted in instructions for this fictional design.

Explore the tool

```
python3 weekly/challenges/week-006-fictional-isa.py table  
  
python3 weekly/challenges/week-006-fictional-isa.py \  
    encode add r1 r2 r3  
  
python3 weekly/challenges/week-006-fictional-isa.py \  
    encode addi r1 r2 -5  
  
python3 weekly/challenges/week-006-fictional-isa.py decode 0x1298  
  
python3 weekly/challenges/week-006-fictional-isa.py repl
```

For each encoding, cover the hexadecimal output and predict it by hand first. Then split the 16-bit result into fields and explain every bit.

Required encoding exercise

Encode these without the tool, then verify:

```
add  r5 r2 r7  
sub  r1 r1 r3  
addi r4 r0 -12  
ld   r2 r6 20  
st   r3 r6 -8  
beq  r1 r0 -4  
jmp  100  
halt
```

Now decode:

```
0x1688  
0x5a7f  
0x74b8  
0x8203  
0x9ffc  
0xa123  
0xf001
```

Some patterns may be invalid or non-canonical. A robust decoder must not pretend every 16-bit word denotes a normal instruction.

Test the boundaries

Predict whether each command succeeds:

```
python3 weekly/challenges/week-006-fictional-isa.py encode addi r1 r2 31
python3 weekly/challenges/week-006-fictional-isa.py encode addi r1 r2 32
python3 weekly/challenges/week-006-fictional-isa.py encode addi r1 r2 -32
python3 weekly/challenges/week-006-fictional-isa.py encode addi r1 r2 -33
python3 weekly/challenges/week-006-fictional-isa.py encode jmp 2047
python3 weekly/challenges/week-006-fictional-isa.py encode jmp 2048
```

Explain the limits from field width and two's-complement representation, without memorizing the numbers.

Modify one design choice

Choose exactly one:

- expand from 8 to 16 registers;
- expand `ADDI` to a 9-bit signed immediate;
- make all branch offsets byte-based;
- add a `MUL` instruction;
- add byte and word loads alongside the current load;
- add a 32-bit “long instruction” format.

Before editing the script, draw the revised format. State what capability is gained and what is lost. If your fields no longer fit 16 bits, that is not a small implementation inconvenience—it is the design constraint becoming visible.

Part XII — Write the one-page ISA design note

Write one page titled **Cedar-16 ISA Design Note**. This is part of the build evidence, not optional documentation after “the real coding.”

Include these six compact sections:

1. Purpose and scope

What kind of machine is Cedar-16 for? Teaching, a tiny controller, or something else? What is deliberately omitted?

2. Architectural state

State:

- instruction width;
- register count and width;
- program-counter behavior;
- byte- or word-addressed memory;
- any special meaning for `r0`;
- visible flags, if any.

Do not invent answers silently. The current encoder defines fields but leaves some execution semantics for you to design.

3. Instruction formats and table

Draw every format and list opcode, mnemonic, operands, and one-sentence semantics. Specify which bit patterns are reserved.

4. Addressing and control flow

Define:

- load/store address calculation;
- immediate sign extension;
- branch base address;
- branch/jump offset units;
- alignment behavior.

“PC-relative” is incomplete until the PC base and offset scaling are defined.

5. Exceptions and invalid encodings

What happens on:

- a reserved opcode;
- nonzero reserved bits;
- misaligned memory access;
- arithmetic overflow;
- an out-of-range address?

An ISA contract must define failures as carefully as successful operations.

6. Tradeoff and revision

Describe one choice you would change in a second version, including:

benefit gained → encoding/implementation/software cost → compatibility effect

This final section is where you demonstrate that an ISA is a set of connected tradeoffs, not a mnemonic list.

Part XIII — Mastery evidence

Explain

Without notes, answer:

Why can two CPUs implement the same ISA differently?

A complete answer should reconstruct this chain:

1. The ISA defines software-visible encodings, state, operations, and results.
2. It does not prescribe every internal circuit or timing choice.

3. Different implementations can use different pipelines, caches, predictors, execution units, scheduling, and internal operations.
4. Each must preserve the required architectural behavior for supported ISA features.
5. Therefore compatible binaries can run on physically different processors, while performance, power use, and microarchitectural side effects differ.

Do not stop at “because ISA is software and microarchitecture is hardware.” The ISA is not software; it is a specification contract implemented by hardware or sometimes by a software model.

Also explain:

- opcode versus operand;
- register versus memory versus immediate operand;
- fixed- versus variable-length encoding tradeoffs;
- why instruction count does not directly equal cycles;
- ISA versus ABI;
- where the specification, bytes, and decoder each exist.

Draw

On one page, draw:

```

ISA manual/specification
  ↓ guides
assembler/decoder/compiler implementations
  ↓
ELF .text bytes → memory → fetch → decode → internal work
                        ↓
                    architectural state transition

```

Add two different CPU implementations beneath the same ISA specification. Give each different pipeline/cache/internal designs but the same architectural result.

Observe

Record:

- ELF Machine and .text information from readelf;
- one raw x86-64 instruction byte sequence and its decoded form;
- one meaningful GCC-versus-Clang or -O0-versus- -O2 difference;
- one source-level if or switch and the actual generated control sequence;
- one place where assembly syntax could be mistaken for machine bytes.

Build

Complete:

- the Cedar-16 hand encoding and decoding set;
- boundary/error tests;
- one deliberate ISA modification;
- the one-page ISA design note.

Why?

Add short answers to the notebook:

1. Why is an ISA a contract rather than a physical component?
2. Why can a more expressive instruction require a more complex encoding?
3. Why does a load/store ISA make memory traffic explicit?
4. Why can a variable-length ISA improve density but complicate decode?
5. Why is an immediate part of an instruction rather than a register?
6. Why is an ABI not the same thing as an ISA?
7. Why can one x86 instruction become several internal operations?
8. Why can two correct compilers emit different byte sequences?
9. Why is an invalid encoding part of ISA design?
10. Where, exactly, did you observe machine code this week?

Mental model at the end of Week 6

```
software needs stable operations
  ↓
ISA specifies visible state, encodings, behavior, and exceptions
  ↓
assembler/compiler selects instruction encodings
  ↓
ELF .text stores machine-code bytes
  ↓
loader maps bytes into a process
  ↓
CPU fetches and decodes according to its implementation
  ↓
microarchitecture performs internal work
  ↓
architectural state changes as the ISA requires
```

The essential sentence is:

Two CPUs can run the same ISA program because they honor the same architectural contract; they can differ internally because the contract specifies required visible behavior, not one mandatory pipeline, cache, or circuit layout.

References used selectively

Official architecture specifications and current documentation:

- Intel, *Intel® 64 and IA-32 Architectures Software Developer's Manual*, current manuals landing page (version 092 at time of consultation), with Volume 2 covering instruction formats and the instruction reference: <https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>

- Intel, *Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 2*, Chapter 2, “Instruction Format,” and §3.1.1, “Instruction Format”: <https://cdrdv2-public.intel.com/819716/325383-sdm-vol-2abcd.pdf>
- AMD, *AMD64 Architecture Programmer's Manual, Volume 3: General-Purpose and System Instructions*, publication 24594, especially Chapter 1 “AMD64 Instruction Encoding” and Chapter 2 “Instruction Overview.” AMD's current processor documentation identifies publication 24594 as the general-purpose/system instruction manual: https://docs.amd.com/api/khub/documents/HwPCwoUzi4L8X5f_1Khe9A/content
- RISC-V International, *The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture*, current ratified specification index, especially “RV32I Base Integer Instruction Set,” “RV64I Base Integer Instruction Set,” and the compressed-instruction extension: <https://docs.riscv.org/reference/isa/unpriv/unpriv-index.html>

Local companion reading:

- Randal E. Bryant and David R. O'Hallaron, *Computer Systems: A Programmer's Perspective*, 3rd ed., Chapter 3, §§3.1–3.4, “A Historical Perspective” through “Accessing Information,” book pages 165–211. Local catalog entry: [source-materials/library/books/01-foundations-and-architecture/computer-systems-programmers-perspective-bryant-ohallaron-3e-global.pdf](#)
- Yale N. Patt and Sanjay J. Patel, *Introduction to Computing Systems*, 2nd ed., Chapter 4, §§4.1–4.3 on the von Neumann model and instruction processing, and Chapter 5, §§5.1–5.3 on the LC-3 ISA and instruction set, book pages 81–150. Local catalog entry: [source-materials/library/books/01-foundations-and-architecture/introduction-to-computing-systems-patt-patel-2e.pdf](#)

Next bridge: once instructions can encode arithmetic, memory access, and next-PC choices, the CPU can form branches and loops. That raises a deeper question: how does normal execution stop for exceptions, interrupts, and privileged operating-system control?